

University of Barishal



A Report on
"Safe Plane Landing"

Graphics Design & Construction Lab

Course Code: CSE-4110

Submitted to:

Dr. MD. Manjur Ahamed

Associate Professor
Department of Computer Science and
Engineering
University of Barishal

Submitted by

MD.Emon & Shariful Islam

Roll No: 21CSE027 & 21CSE049

Year: 4th

Semester: 1st

Session: 2020-21

Department of Computer Science and
Engineering

University of Barishal

Date: 18 June 2026

Abstract

This project presents a 3D airport simulation developed using OpenGL and GLUT. The scene includes an animated airplane, runway, airport terminal, control tower, trees, clouds, mountains, and a city skyline to create a realistic airport environment. The program supports keyboard and mouse input for controlling the camera, aircraft movement, door operation, takeoff, landing, and day-night switching. The project demonstrates key computer graphics concepts such as 3D object construction, transformation, animation, lighting, color management, and interactive user input. It is suitable as a student-level graphics application because it combines visual realism with simple control logic and an engaging animated environment.

Introduction

Computer graphics is the field of creating, manipulating, and displaying visual content using computers. It is widely used in simulation, games, visualization, design, education, and entertainment. In this project, the aim is to build an interactive airport environment that demonstrates the practical use of OpenGL functions and basic 3D rendering techniques.

This airport simulation shows how geometric primitives can be combined to build realistic objects such as aircraft, buildings, trees, and landscape elements. It also demonstrates animation through continuous updates and user interaction through mouse and keyboard handling. The final result is not only visually appealing but also educational, since it connects theoretical graphics concepts with actual implementation.

Problem Statement

The objective of this project is to design and implement a 3D airport scene that is free from errors and capable of showing interactive animation. The system should allow the user to move the camera, observe the aircraft from different angles, and control airplane functions using keyboard input.

The project must satisfy the following goals:

- Create a complete 3D airport environment.
 - Include aircraft animation with landing and takeoff behavior.
 - Implement camera movement using mouse and keyboard controls.
 - Add realistic lighting, color variation, and environmental details.
 - Demonstrate core OpenGL transformation functions such as rotation, translation, and scaling.
-

Objectives

The main objectives of the project are:

- To demonstrate the concept of computer graphics through a complete OpenGL scene.
 - To build 2D and 3D objects using primitive shapes such as cubes, spheres, cylinders, and disks.
 - To implement keyboard and mouse interactions for controlling the scene.
 - To animate the airplane, clouds, radar, and environmental effects.
 - To apply lighting and day-night switching for improved visual quality.
 - To create a project that matches academic course outcomes CO1, CO2, and CO3.
-

Tools and Technologies

The project uses the following tools and technologies:

Tool/Technology	Purpose
C Language	Core programming language
OpenGL	3D graphics rendering
GLUT	Window management and input handling
GLU	Utility functions for cylinders, disks, and perspective
Math Library	Trigonometric calculations for animation
Standard C Library	Basic program support

OpenGL is suitable for this project because it provides direct access to rendering functions and transformation operations. GLUT is used to create the window, manage input, and run the main event loop.

Program Structure

The program is organized into several major parts:

- Global variables for camera position, airplane movement, lighting state, and animation parameters.
- Drawing functions for scenery such as sky, sun, clouds, mountains, skyline, trees, runway, and fence.
- Drawing functions for airport terminal, control tower, airplane body, engines, wings, landing gear, stairs, and door.
- Animation update function to handle movement and rotation.
- Input handlers for keyboard, mouse, special keys, and window resizing.
- Initialization and main function for OpenGL setup and application execution.

This structure is clean and modular, which makes the program easier to understand, extend, and debug.

Scene Design

The airport scene is designed as a complete environment rather than a single object. The background includes sky color variation, mountains, clouds, sun, and skyline buildings. The middle area contains the airport terminal building and control tower, while the lower area includes the runway, fence, and ground surface.

The airplane is placed as the main animated object. The design also includes environmental objects like trees and clouds so the scene feels alive. The composition gives depth by placing objects at different z-levels and using perspective projection. This helps demonstrate 3D scene organization in a practical way.

3D Object Modeling

The objects in the project are made from basic OpenGL primitives. Cubes are used for buildings, windows, runway markings, and fence segments. Cylinders are used for the control tower, tree trunks, and airplane engines. Spheres are used for the sun, clouds, tree foliage, and plane nose.

This method is important in computer graphics because complex objects are often built from simpler shapes. By scaling, translating, and rotating these primitives, the program forms a detailed airport scene. This directly supports CO2 because it constructs 2D and 3D graphics through graphics programming.

Transformations Used

The project makes extensive use of transformation functions to position and shape objects.

Translation

Translation is used to move objects to their correct positions in the scene. For example, the airplane is translated to `planeX`, `planeY`, and `planeZ`, while clouds and trees are also placed at different coordinates.

Rotation

Rotation is used for many parts of the scene:

- The camera can rotate around the scene.
- The airplane is rotated to face its direction.
- The radar rotates continuously.
- The plane door rotates when opened.
- Wings, fins, and tower parts use rotation to align correctly.

Scaling

Scaling is used to adjust object size:

- Buildings are stretched into tall structures.
- Runway lights are flattened.
- Airplane parts are scaled to form fuselage, wings, and tail.
- Trees use scaled cylinders and spheres for natural proportions.

These transformation functions are the core of 3D graphics modeling and are clearly demonstrated in the project.

Animation System

The animation is controlled by the `update()` function, which runs repeatedly using a timer. This function updates the radar angle, cloud movement, sun position, airplane fan speed, door motion, and landing/takeoff states.

The airplane animation is especially important. During landing mode, the aircraft moves toward the runway and gradually decreases altitude. During takeoff mode, it increases speed, rises upward, and leaves the runway. The engine fan rotates continuously according to the current fan speed, which gives the impression of motion and power.

The animation is suitable with the environment because each moving object behaves consistently with the airport setting. This fulfills the CO2 requirement for suitable animation implementation.

Camera and Interaction

The project includes a flexible camera system. The user can orbit the camera around the scene by dragging the mouse. The camera height can also be adjusted with the mouse, while zooming is possible using the mouse wheel or keyboard keys.

The special arrow keys modify the camera angle and height. This makes the scene more interactive and allows the viewer to observe the airport from multiple viewpoints. Such camera control is useful in graphics applications because it improves scene inspection and user experience.

The interaction also supports CO3 because the user can respond to the program's control instructions and receive feedback through the scene changes. The simulation behaves as an interactive visual system rather than a static image.

Keyboard Controls

The following keyboard functions are implemented:

Key	Function
<code>L / l</code>	Start landing mode
<code>T / t</code>	Start takeoff mode
<code>W / w</code>	Increase flap angle

S / s	Decrease flap angle
F / f	Increase fan speed
G / g	Decrease fan speed
O / o	Open the aircraft door
C / c	Close the aircraft door
R / r	Toggle day/night mode
+ / =	Zoom in
-	Zoom out
Esc	Exit the program

These controls make the simulation interactive and demonstrate keyboard event handling in OpenGL.

Lighting and Color Mapping

Lighting is enabled using `GL_LIGHTING` and `GL_LIGHT0`. The scene includes ambient and diffuse light settings to make the objects visible and more realistic. Color values are changed according to day and night mode so that the environment adjusts visually.

The project also uses color mapping for different components:

- Blue glass windows for the terminal in daytime.
- Warm yellow tones at night.
- Gray buildings and runway surfaces.
- Green vegetation for trees.
- White clouds in daytime and darker clouds at night.

These choices help the scene look natural and suitable for an airport environment. The color changes also demonstrate visual adaptation based on the current mode.

Texture Mapping Note

The project mainly uses color-based surface styling rather than image texture files. However, it still demonstrates the idea of texture/color mapping by assigning appropriate colors to different objects to simulate material variation.

For example:

- Glass areas use blue tones.
- Metallic airplane parts use light gray tones.
- Dark runway and asphalt use muted gray-black colors.
- Warning lights and windows use bright shades for visibility.

If texture mapping is required in future improvement, image-based textures can be added to the runway, terminal walls, or aircraft body. That would make the scene even more realistic.

Airplane Functionality

The airplane is the central object of the project. It includes a fuselage, wings, tail, engines, landing gear, windows, and movable door. The fan blades rotate continuously to simulate engine motion. The landing gear remains visible during landing and ground operations.

The aircraft can switch between landing and takeoff modes. When landing, it descends slowly and aligns with the runway. When taking off, it speeds up and gains altitude. This logic gives the simulation a dynamic and realistic behavior.

Airport Environment

The airport environment is made up of several supporting objects:

- The terminal building provides the main airport structure.
- The control tower adds realism and supports airport identity.
- The radar on top of the tower rotates to represent monitoring activity.
- The runway lights guide aircraft movement.
- The fence creates a boundary around the airport area.
- Trees and skyline buildings enhance the background environment.

This full environment is important because the project is not only about airplane animation but also about scene composition and environmental storytelling.

Day and Night Mode

The project includes a day-night toggle controlled by the **R** key. During daytime, the sky is bright, the sun is high, and the buildings appear in lighter colors. During nighttime, the sky darkens, the sun moves down, and lights on the runway and buildings become more visible.

This feature improves visual variety and demonstrates environmental state management. It also shows how color and lighting can be used to create different moods within the same graphics program.

Testing and Validation

The project was tested by checking different parts of the program separately and then as a whole. The following points were verified:

- The window opens correctly.
- The scene renders without errors.
- Mouse controls rotate the camera correctly.
- Keyboard input changes airplane states properly.
- Landing and takeoff animations work as expected.
- Door opening and closing functions operate correctly.
- Day and night mode changes the scene colors.

Testing showed that the program is stable and suitable for demonstration purposes. The code is organized so each feature can be observed independently and as part of the full simulation.

Results

The final result is a working 3D airport simulation with interactive control. The scene successfully displays an airport environment with an animated airplane, moving clouds, rotating radar, camera orbit, day-night switching, and airport structures.

The project demonstrates:

- 3D modeling.
- OpenGL transformations.
- Lighting and coloring.
- Animation loops.
- Keyboard and mouse handling.

- Scene composition.

Overall, the application meets the major requirements of CO1, CO2, and CO3. It also provides a strong example of how computer graphics concepts can be applied in a real program



Limitations

Although the project is effective, it has some limitations:

- It uses fixed-function OpenGL rather than modern shader-based graphics.
- The environment is built from simple primitives, so realism is limited.
- No actual image textures are used.
- The aircraft movement follows scripted logic instead of physics-based flight modeling.
- Sound effects are not included.

These limitations are normal for a student-level computer graphics project and can be improved in future versions.

Future Improvements

Several improvements can be made to enhance the project:

- Add real image textures to buildings, runway, and airplane surfaces.
- Use modern OpenGL shaders for advanced rendering.
- Include realistic shadow effects.
- Add sound for engine, landing, and environment.
- Improve airplane physics and flight behavior.
- Add moving airport vehicles, passengers, or weather effects.

These enhancements would make the simulation more realistic and visually rich.

Conclusion

This project successfully demonstrates the concept of computer graphics through a 3D airport simulation built with OpenGL and GLUT. It combines object modeling, transformations, lighting, animation, and interactive camera control into one working system. The program fulfills the intended course outcomes by showing both technical graphics implementation and user interaction.

The project also proves that complex-looking scenes can be created from simple geometric primitives and well-planned animation logic. It is a strong example of how computer graphics technology can be used to build educational and interactive applications.

Rubric

****** Should be attached with your report for marking purpose**

Course Outcome	Notes	Marks
CO1: Demonstrate the concept of computer graphics and ability to use the computer graphics technology.		25 marks
<u>Overall program structure:</u> - Free from error	5 marks	
<u>OpenGL Function:</u> - Apply functions of keyboard handling - Rotation - Translation - Scaling X, Y, and Z Direction <u>Texture Mapping:</u> - Apply functions of texture/color mapping for appropriate color	15 marks 5 marks	
CO2: Construct 2D and 3D graphics by implementing concepts of computer graphics and computer graphics programming.		25 marks
<u>Object:</u> -All required objects are provided	10 marks	
<u>Animation:</u> - Animation is implemented and suitable with the environment	10 marks	
<u>Camera / Lighting:</u> -Appropriate technique is implemented and suitable with the environment	5 marks	
CO3: Respond to instruction by listening actively and give feedback		25 Marks
-Camera moving -Animation -Answer the questions regarding basic of computer graphics	10 marks 10 marks 5 marks	

CO4: Work together effectively to achieve the same goal by building a good relationship and interaction among team members.			25 M ar ks
Student 1 Name: ID:	Objects (% of contribution) =	5 Mark s	
	Camera (% of contribution) =	5 Mark s	
	Animation (% of contribution) =	5 Mark s	
	Report (% of contribution) =	10 Mark s	
Student 2 Name: ID:	Objects (% of contribution) =	5 Mark s	
	Camera (% of contribution) =	5 Mark s	
	Animation (% of contribution) =	5 Mark s	
	Report (% of contribution) =	10 Mark s	